

Early Boot and MMU Enable

On reset an ARM64 CPU starts executing from a fixed physical address (often in onboard ROM or bootloader). Early boot firmware (like ARM Trusted Firmware/BL1, BL2, U-Boot, etc.) runs with the MMU off, using physical addressing. Until the MMU is enabled, all code and data references are by physical addresses, so early code must be linked and run at fixed addresses or with **identity mapping**. For example, the Linux `head.S` sets up an initial page table that *identity-maps* the kernel's own image and RAM. In particular, it creates a minimal page-table in memory (e.g. `idmap_pg_dir`) that maps the physical range of the kernel into the same virtual addresses. At this point registers like **TTBR0_EL1**/**TTBR1_EL1** (the base pointers for level-0 page tables) and **TCR_EL1** (translation control) are still unset or empty, and **SCTLR_EL1** (System Control Register) has the MMU disabled (M=0).

Once the kernel has set up these tables, it enables the MMU with a sequence like (pseudocode):

```
msr ttbr0_el1, <phys-to-ttbr>(idmap_pg_dir) // point TTBR0 at the identity-map
PGD
msr ttbr1_el1, <phys-to-ttbr>(swapper_pg_dir) // point TTBR1 at the kernel
swapper_pg_dir
msr tcr_el1, <T0SZ,T1SZ config> // set up address size, granule,
cacheability
msr sctlr_el1, SCTLR_MMU_ON | ... // set M=1 to enable MMU, caches,
etc.
isb // ensure all instructions see new
mapping
```

Here **TTBR0_EL1** typically points at user-space tables (when running user tasks), and **TTBR1_EL1** to kernel-space tables. The Linux head code passes the physical address of the ID-map table to the `__enable_mmu` routine, which writes `ttbr0_el1` / `ttbr1_el1` and then sets `sctlr_el1` (M=1) to enable translation ¹ ². After the `isb`, the CPU automatically starts interpreting all instruction and data addresses through the MMU. At this moment the CPU “sees” virtual addresses: the low half (MSB=0) uses TTBR0, the high half (MSB=1) uses TTBR1. Since our ID map was 1:1 for the kernel's own addresses, the running code continues seamlessly but now in *virtual* mode.

Internally, once the MMU is on the layout of memory changes. All future instruction fetches and data accesses go through the translation tables, and the CPU will raise *translation faults* if it encounters an address not mapped or with insufficient permissions. Meanwhile the identity-mapped region ensures that the already-loaded kernel image and some low-level data structures remain valid. From this point on, almost all kernel code runs with virtual addressing.

Virtual-to-Physical Address Translation

With the MMU on, every memory access is a *virtual address* (VA). The ARM64 MMU breaks a 64-bit VA into fields that select page-table entries and an in-page offset. For example, with 4 KB pages and a 4-level translation (TTBR0/1 + 3 levels of tables), the VA bits are partitioned roughly as follows (from most-significant bit down):

```
| 63 . . . 56 | 55 . . . 48 | 47 . . . 39 | 38 . . . 30 | 29 . . . 21 | 20 . . . 12 |
11 . . . 0 |
| Unused bits | Unused bits | L0 Index | L1 Index | L2 Index | L3
Index | Offset |
| (must be 0 ) | or sign-ext | (9 bits) | (9 bits) | (9 bits) | (9
bits) | 12 bits |
| (selects TTBR) | (selects TTBR) |
```

Bit 63 (the MSB) selects whether to use **TTBR0** or **TTBR1** (0 for TTBR0 = user space, 1 for TTBR1 = kernel space). Each subsequent 9-bit field indexes into a page table at that level. The low 12 bits are the in-page offset (since 4 KB pages give 12-bit offset). Thus an AArch64 page-table walk goes: take TTBRx, then use VA[47:39] to index the Level-0 table (sometimes called PGD), VA[38:30] to index Level-1 (PUD), VA[29:21] for Level-2 (PMD), and VA[20:12] for Level-3 (PTE). The selected PTE provides the final physical page base (bits 47:12 of the PA) and attributes; the offset [11:0] is added to produce the final physical address.

Each table entry is 64 bits and has bits to indicate *valid*, *block vs table*, physical base, and access attributes. In general, **bit[0] = 1** means the entry is valid; **bit[1] = 0** indicates a *block/page descriptor* (an actual mapping), and **bit[1] = 1** indicates a *table pointer* to the next level ³. For 4 KB granularity, a Level-1 block maps a 1 GB region (bits[47:30] of PA), a Level-2 block maps 2 MB (bits[47:21] of PA) ⁴, and a Level-3 entry (always a *page* entry) maps 4 KB (bits[47:12] of PA in the descriptor). A *table* descriptor (L0-L2) has bits[47:12] = address of the next-level table (aligned), and bits[11:0]=0 ⁵. In software terms, Linux's `pgd`, `pud`, `pmd`, `pte` structures correspond to these levels (Linux may fold L0/L1 or skip unused levels depending on VA width).

ARM64 also supports larger pages: for example, a 4 KB granule can use a Level-2 block (2 MB page) or even Level-1 block (1 GB page). Linux often uses 2 MB pages (via `PAGE_SIZE_2MB`) for the kernel text, etc. Smaller granules (16KB/64KB) and fewer levels also exist, but the principle is the same: walk each level until either a block entry or the final page entry is found.

These descriptor entries encode not only the physical address but also permissions and attributes. The descriptor fields include **AP bits** (access permissions) that indicate read/write and user/kernel permission, **UXN/PXN** bits (user/privileged eXecute Never) to disable execution, an **Access Flag (AF)** to indicate if a page has been accessed, cacheability attributes (with an MAIR index), shareability bits, and dirty bits. For example, a typical Linux kernel PTE will set UXN=1 (forbid user execution) and PXN=1 (forbid privileged execution on some blocks), and set R/W or X depending on the page (text pages are RX, data pages RW).

At run-time, the **hardware page walker** automatically performs this multi-level lookup on a TLB miss or first access. If at any level the table entry is *invalid* or not present, the MMU generates a *Translation Fault*

exception ³. If it finds a block or page entry, it combines the output physical base with the offset and returns that as the real physical address, also applying any memory attribute (cache) policies from the entry.

Example: Suppose we have VA = `0xffff_ffff_8000_1234` (a kernel VA on a 48-bit system). Bit63=1 → TTBR1 is used. Bits47:39 index into the top-level (PGD) table; assume that entry is valid and points to a level-1 table. Bits38:30 index that table; if that entry is a block descriptor (say mapping 1 GB), then the hardware can stop here and form PA = (entry.PA[47:30] << 30) | (remaining VA bits). Otherwise it proceeds to level-2 and level-3 similarly. In Linux, low-level code (e.g. dumpers or `TRANSLATE()` macros) often illustrate this walk.

Kernel Virtual Memory and Physical Memory Management

Once the MMU is on, the Linux kernel runs almost entirely with virtual addresses. The kernel code and data are located in the high half of the VA space (with bit63=1) and are globally mapped in a single “swapper” page table (`swapper_pg_dir`). User tasks run in the low half (bit63=0) with user mappings in TTBR0. Crucially, Linux creates a **direct linear mapping** of all physical RAM into the kernel’s address space (often called the *physmap*). In other words, every physical page is also mapped at some fixed high virtual address (e.g. starting at `PAGE_OFFSET`). This means that the kernel can convert between virtual and physical addresses easily by arithmetic: `virt_to_phys(v) = (v - PAGE_OFFSET) + PHYS_OFFSET` if `v` is in the linear map ⁶ ⁷. Here `PHYS_OFFSET` is usually zero or the low physical address of RAM. The macros `__virt_to_phys()` and `__phys_to_virt()` in `arch/arm64/include/asm/memory.h` implement this: they check if a kernel address is in the linear-mapped region (`__is_lm_address`) and then adjust by `PAGE_OFFSET` and `PHYS_OFFSET` ⁶ ⁷. (Note: on relocatable kernels a `kimage_voffset` may shift this mapping, but the idea is the same.)

Because of this physmap, the kernel never needs to use raw physical addresses in its code; it simply uses the corresponding virtual addresses. The only place a driver or code might need a physical address is for programming a DMA engine or MMIO register, but even then Linux provides DMA mapping APIs to do that safely. The kernel *cannot* simply dereference an arbitrary physical address; it must go through either the physmap or create explicit mappings. For example, the function `phys_to_virt(paddr)` in code yields a void* pointer by doing `(paddr - PHYS_OFFSET) | PAGE_OFFSET` ⁷.

Underneath these virtual mappings, Linux tracks free/used physical memory with **struct page** metadata. Each physical page frame has a `struct page` entry (often in a global `mem_map` or, in sparsemem, per-node arrays) that holds flags and counters about that page (free, reserved, mapped, etc.). Memory is managed by the *buddy allocator* and the kernel’s page allocator, which use these `struct page` entries. The kernel’s memory model on ARM64 is typically *sparse* (section-based), meaning it only allocates `struct page` arrays for actual memory sections present, rather than a flat array for all possible PFNs ⁸.

In short, after MMU enable the kernel “lives” in the virtual world. It uses virtual pointers (and the linear physmap) everywhere, and it keeps a separate level of bookkeeping for physical pages via `struct page`. Functions like `virt_to_phys(vaddr)` and `phys_to_virt(paddr)` bridge between them when

needed. Conversion functions `__pa()` and `__va()` (macros) implement these via simple arithmetic and are inline. The kernel never actually loads or manipulates addresses as pure physical except in a few low-level places (for example, early boot or debug code).

Page Fault Handling

Whenever a CPU access (fetch or load/store) encounters a VA without a valid translation, the MMU generates an exception. On ARM64 this is a **Data Abort** (for loads/stores) or **Instruction Abort** (for fetches) with an EC (Exception Class) and ISS (Instruction Specific Syndrome) in the **ESR_EL1** register, and the faulting address in **FAR_EL1**. In Linux's ARM64 exception vectors, all user/kernel page faults end up calling a generic fault handler. If the fault occurred in user mode (EL0), it will eventually call `do_page_fault()`. If in kernel mode (EL1), a kernel Oops occurs (`__do_kernel_fault()`).

Inside `do_page_fault`, the kernel determines why the page was missing and what to do. It uses `FAR_EL1` as the faulting VA and looks up the current process's `mm_struct`. It finds the `vm_area_struct` (VMA) covering that VA. If no VMA exists or the access (read/write/exec) is not allowed, a SIGSEGV is sent to the process. Otherwise, if the VMA is valid (e.g. an anonymous or file mapping that permits the access), the kernel must resolve it.

For **anonymous pages** (heap, stack, or `mmap` with `MAP_ANONYMOUS`), Linux typically uses *demand paging*. If the VMA has no page table yet, or the PTE has `PFN=0`, the handler will allocate a new physical page (via `alloc_page()`) and map it. The process is “killed lazily”; that is, Linux does *optimistic allocation* (overcommit): it records the allocation in memory accounting, but the physical page isn't provided until first touch. Thus the actual mapping into RAM happens now. This is why `malloc()` or `mmap()` appear to succeed quickly – they don't actually provision pages until the fault. Remus Rusanu summarizes: “Linux does deferred page allocation... The memory you get back from malloc is not backed by anything and when you touch it ... a fault will occur, the kernel will ... find an unused physical 4KB of memory, update the PTE, ... then return to the application”⁹. If no memory is available (and no swap), the kernel will OOM and kill a process at this point.

On a **copy-on-write** fault (e.g. write to a shared page after `fork()`), the existing page is remapped as read-only, and the fault is treated as write to a read-only page. Linux's fault handler sees a write with `VM_SHARED` cleared, allocates a new page, copies the contents, and installs the new page for the process. It then invalidates the old TLB entry and resumes execution. Page faults for file-backed VMAs (like `mmap` of a file) may involve reading data from disk if not already cached. Minor faults (page already in RAM but not mapped in this process) only require installing the existing page; major faults involve I/O.

After allocating or looking up the page, the kernel writes the new PTE and usually performs a *TLB invalidate* for that VA (using `tlbi` instructions) to ensure the new mapping is used. Then it returns from the exception, and the faulting instruction is retried—this time succeeding with the physical page now present.

In ARM64, the fault reason is encoded in `ESR_EL1`'s FS (Fault Status) field. Common faults are *translation fault* (no entry) or *permission fault* (entry present but bad access). The kernel copies `ESR_EL1` to `thread.fault_code` for accounting. `FAR_EL1` gives the VA. (On data abort from an instruction, `FAR_EL1` holds that VA.) In summary: **CPU access → TLB miss / invalid PTE → fault; then kernel handler → allocate/map page and update PTE → flush TLB → retry instruction.**

TLB and Its Impact

A Translation Lookaside Buffer (TLB) is a hardware cache of recent VA→PA translations. Every memory access first probes the TLB. On a **TLB hit**, the PA and attributes are returned immediately. On a **TLB miss**, the MMU must walk the page table (as above) to find or create a translation. This is much slower (several memory accesses), so the TLB is crucial for performance. ARM64 typically has separate L1 TLBs for instructions (I-TLB) and data (D-TLB) at EL0/EL1, and often larger L2 unified TLBs.

Each TLB entry is tagged with an Address Space ID (ASID) so that entries for different processes (different TTBR0) don't clash. When the kernel context-switches to a new process, it changes the ASID register (and TTBR0 to the new process's page table) so the old entries stay in the TLB but become "invisible" since their ASID no longer matches. This avoids flushing the entire TLB on each switch. If the ASID space wraps or we switch to the same ASID, Linux will do a TLB invalidate (via the `tlbi` instruction).

On SMP systems, when one CPU changes a shared page table (e.g. freeing a page or changing permissions), it must perform a TLB shutdown: sending IPIs to other CPUs to invalidate that VA/ASID from their TLBs (using `TLBI` instructions). ARM64 provides instructions like `TLBI VMALLE1` (invalidate all TLB entries at EL1) or more targeted ones. Linux has helper functions (`__flush_tlb_all()`, `__flush_tlb_range()`) in `arch/arm64/mm/tlbflush.h` for this.

Huge pages (2 MB or 1 GB) reduce TLB pressure because one entry covers many small pages. Network servers often enable `hugepages` for buffers to minimize TLB miss rates. TLB misses hurt packet processing: every packet buffer access might suffer a delay if its VA isn't in the TLB, reducing throughput. That's one reason frameworks like DPDK and XDP encourage using physically contiguous huge-page DMA buffers (so that fewer, larger pages cover more data, and TLB entries last longer).

Caches and Coherency

ARM64 cores have multi-level CPU caches (L1 instruction, L1 data, often L2 per cluster, sometimes L3). Cache lines are typically 64 bytes. An instruction fetch goes to the I-cache (L1I), and loads/stores go to the D-cache (L1D), then possibly L2 etc. These caches speed up memory access by holding recent lines. The system is coherent: if one core writes to a cache line, other cores will see the updated value (via a coherency protocol like MOESI on the CCI interconnect).

False sharing occurs if two threads on different cores use different variables that happen to be on the same cache line; they'll ping-pong the line and slow each other. In packet processing, careful struct alignment (e.g. `sk_buff` fields) avoids this. ARM64 has instruction (`ISB`) and data (`DSB`, `DMB`) barriers. A **DMB** (Data Memory Barrier) ensures that all explicit memory accesses before the DMB complete before any after it begin. A **DSB** is a stronger sync that also waits for cache/TLB maintenance. After turning on the MMU or after writing new page tables, Linux always executes an `isb` (instruction barrier) and `dsb` to ensure the MMU/scheduler sees the new state.

For DMA, coherency matters. If a NIC DMA-writes into main RAM, and the CPU has that line in cache, the CPU might not see the write until the cache is invalidated. Conversely, if CPU writes a buffer that the NIC will read, the CPU must flush (or write-back) those cache lines to memory first. Linux DMA APIs

(`dma_map_single()`, etc.) handle this: they insert cache maintenance instructions (like `DC CVAC` to clean D-cache) so device and CPU see consistent data. Without it, packet reception could get stale or corrupted data.

Cache misses hurt latency: a packet arrival interrupt might incur random D-cache misses when accessing the `sk_buff` or page data. Networking code often prefetches or uses cache-aligned data structures to minimize misses. NUMA effects also arise: if the NIC is on node 0 and it DMA into memory on node 1, the CPU on node 0 accessing that memory will face a remote access penalty. Linux network drivers and `numactl` / `numabind` can try to pin packet buffers on the same NUMA node as the CPU/NIC.

Userspace Memory Allocation (`malloc`)

In user space, calls like `malloc()` or `new` do not immediately allocate physical RAM. Typically, small allocations come from the process's heap (managed by `brk` / `sbrk`), while large allocations use `mmap(MAP_ANONYMOUS)`. In either case, Linux uses lazy allocation. That is, `malloc(...)` essentially reserves a range of *virtual addresses* (expanding the heap or creating a VMA for an `mmap`), but does **not** populate page tables with real pages right away.

When the program first touches those pages (e.g. writes to them), a page fault occurs. The kernel's page-fault handler then allocates actual physical pages. As noted earlier, Linux is "optimistic" about memory; it lets `malloc` succeed even if RAM+swap might not suffice. When a page fault happens on an anonymous page, Linux will finally call the page allocator and attach the page. Remus Rusanu explains: "*Linux does deferred page allocation... The memory you get back from malloc is not backed by anything and when you touch it ... the kernel will ... find an unused physical 4KB of memory, update the page table entry ... and return.*"⁹. If the system is out of memory at that point, it may kill the process (OOM).

Glibc's allocator manages arenas and bins for small chunks. If a requested size exceeds a threshold (~128 KB or so), glibc may use `mmap` instead of `brk`¹⁰. In any case, until the program accesses a page, Linux often maps it to a global **zero page** (read-only mapping to a page of zeros) and sets it copy-on-write. On first write, the fault handler copies the zero page into a new real page (this is an optimization; Linux might just allocate directly, but some kernels historically used a shared zero page for read-only zero-mapping).

Memory fragmentation occurs when many allocations/free patterns leave holes. The glibc malloc arena system tries to reuse freed chunks, but large unused areas may remain. Overcommit can lead to situations where `malloc` returns a pointer but underlying memory may later fail to allocate. The standard advice is to use `calloc` or manually touch pages if you want to pre-fault them. (Linux also offers `MAP_POPULATE` or `mlock()` to pre-fault pages, though not commonly used.)

In summary: **malloc** → (expands address space via `brk/mmap`) → **no physical pages yet** → first access to each page → **page fault** → kernel allocates physical page, zeroes it, updates PTE → TLB reload → program continues⁹.

Kernel and Networking

All the above concepts tie directly into networking performance. When the kernel receives a packet (e.g. via a NIC interrupt), it allocates an `sk_buff` (socket buffer) or similar to hold the packet. An `sk_buff` often includes pointers into a data buffer, which might be a physically-contiguous page (for jumbo frames) or a fragment. To receive packets efficiently, drivers use **DMA mapping**: they allocate pages (sometimes using the **page_pool** allocator optimized for networking) and map them for device use. The `page_pool` API (in `net/core/page_pool.c`) provides fast allocation of page-sized buffers for RX/TX with minimal overhead¹¹. These pages are returned to the driver when the packet is consumed (zero-copy).

When preparing a packet to send, the driver might `dma_map_single()` the `sk_buff`'s data. Under the hood this flushes CPU caches for that memory range (if needed) and programs the IOMMU or device so the NIC sees the correct physical addresses. Upon receiving, the NIC writes into physical memory; the driver later calls `dma_sync_single_for_cpu()` to invalidate the relevant D-cache lines before the CPU reads the packet data. This ensures cache coherency with DMA. If an IOMMU/SMMU is present (common in ARM64 SoCs), the driver also programs it so that the device uses IOVA addresses; this can add another TLB (I/O TLB) lookup in the IOMMU.

Performance implications are key: **TLB misses** on packet buffers cost extra cycles, so using hugepages (as in DPDK) or the Linux `page_pool`'s page reuse (which keeps buffers “hot” in the TLB) is beneficial. **Cache misses** on `sk_buff` headers or packet data also stall CPUs; hence `sk_buff` is designed with cache-line alignment and cache-friendly access patterns (see the `SKB_DATA_ALIGN` macros in `skbuff.c`). False sharing (e.g. two CPUs writing to adjacent fields in a shared structure) can reduce throughput. On NUMA machines, binding network IRQs and memory allocations to the same node avoids remote memory access penalties.

Modern Linux networking features exploit these memory tricks: **XDP/eBPF** can allocate packets directly into `page_pool` pages and reuse them without copying, relying on the page lifecycle and TLB caching. **Zero-copy** APIs (like `sendpage()`) let user-space network stacks hand pages to the NIC with `mmap'd` pages. **DPDK** bypasses the kernel, using hugepage memory and polling to eliminate OS overhead, but it still depends on understanding TLB and cache (userspace libs manage their own hugepage pools and flush caches via instructions or CPU affinity).

In summary, from CPU reset through virtual memory up to `malloc()` and packet I/O, ARM64 Linux tightly couples hardware and kernel logic. The CPU starts in physical mode, switches to virtual mode via a carefully-built identity-map and registers (TTBR0/1, TCR, SCTLR)¹². Every access thereafter uses the MMU's page-table walks (multilevel lookup with descriptors) to translate VA→PA³¹². The kernel itself runs with VA and manages physical RAM underneath via the `physmap` and `struct page`. Misses trigger page faults, which the kernel handles by allocating pages and updating tables. The TLB caches recent translations to speed memory accesses, and CPU caches cache the data. All of these layers (TLB, caches, coherency, memory allocators) together determine the performance of everything — including critical tasks like packet processing. By the end of this chain, a user `malloc()` ends up as virtual pages that only get real RAM on first touch⁹, and networking code carefully choreographs DMA with caches and IOMMUs to move data to/from the NIC.

Sources: ARM ARM and architecture documentation ³ ⁴ ; Linux ARM64 source (`head.S` , MMU code) ¹ ² ; Linux memory model docs ¹³ ; kernel include files ⁶ ⁷ ; page fault/alloc explanations ⁹ ¹¹ ; etc. Each source is cited for key facts above.

¹ ² `head.S` source code [linux/arch/arm64/kernel/head.S] - Codebrowser

<https://codebrowser.dev/linux/linux/arch/arm64/kernel/head.S.html>

³ ⁴ ⁵ D4.3.1 VMSAv8-64 translation table level 0 level 1 and level 2 descriptor formats · ARM Architecture Reference Manual for ARMv8-A

https://armv8-ref.codingbelief.com/en/chapter_d4/d43_1_vmsav8-64_translation_table_descriptor_formats.html

⁶ ⁷ linux/arch/arm64/include/asm/memory.h at master · torvalds/linux · GitHub

<https://github.com/torvalds/linux/blob/master/arch/arm64/include/asm/memory.h>

⁸ How does the Linux kernel initialize the mem_map array on 64 bit systems? - Unix & Linux Stack Exchange

<https://unix.stackexchange.com/questions/562629/how-does-the-linux-kernel-initialize-the-mem-map-array-on-64-bit-systems>

⁹ ¹⁰ Does malloc lazily create the backing pages for an allocation on Linux (and other platforms)? - Stack Overflow

<https://stackoverflow.com/questions/911860/does-malloc-lazily-create-the-backing-pages-for-an-allocation-on-linux-and-oth>

¹¹ Page Pool API — The Linux Kernel documentation

https://docs.kernel.org/networking/page_pool.html

¹² AArch64 Kernel Page Tables - Wenbo Shen(申文博)

<https://wenboshen.org/posts/2018-09-09-page-table>

¹³ www.kernel.org

<https://www.kernel.org/doc/Documentation/arm64/memory.txt>